Übungsblatt 3

Mathematik in Lean

Skript: Abschnitte 3.6–3.8

Vorlesung 3 hat die Taktiken am Beispiel der Mengen vorgeführt und gezeigt, wie Mathlib die Strukturen der Mathematik aus Typklassen aufbaut. Dieses Blatt übt beides: Mengen als Prädikate, Matrizen und lineare Abbildungen mit ihrem Kern, Induktion – und die Frage, was der Typcheck überhaupt zulässt. Teil B formalisiert Teil A, jetzt mit dem Taktikmodus.

Teil A: Auf Papier

Aufgabe A1 (Mengen sind Prädikate)

Es seien α ein Typ und s, t, u Teilmengen von α, in Lean `s t u : Set α`. Wie in der Vorlesung bedeutet $x \in s \cap t$ dasselbe wie $x \in s \wedge x \in t$, $x \in s \cup t$ dasselbe wie $x \in s \vee x \in t$ und $x \in s^{\mathrm{c}}$ dasselbe wie $\neg(x \in s)$. Ferner heißt $s \subseteq t$, dass aus $x \in s$ stets $x \in t$ folgt, und zwei Mengen sind gleich, wenn sie dieselben Elemente haben. Beweisen Sie die folgenden Aussagen, indem Sie sie für ein festes x auf Aussagenlogik zurückführen, und nennen Sie jeweils das aussagenlogische Gesetz, das dabei herauskommt.

(a) $s \cap (t \cup u) = (s \cap t) \cup (s \cap u)$.

(b) Aus $s \subseteq t$ folgt $t^{\mathrm{c}} \subseteq s^{\mathrm{c}}$.

(c) $(s \cap t)^{\mathrm{c}} = s^{\mathrm{c}} \cup t^{\mathrm{c}}$. Beweisen Sie die beiden Inklusionen einzeln. Eine davon braucht den Satz vom ausgeschlossenen Dritten, die andere nicht – welche ist welche? Braucht ihn die Regel $(s \cup t)^{\mathrm{c}} = s^{\mathrm{c}} \cap t^{\mathrm{c}}$ aus der Vorlesung?

Aufgabe A2 (Rechnen mit Matrizen)

Es seien

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad B = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}.$$

(a) Berechnen Sie AB, BA, $\det A$, $\det B$ und $\det(AB)$. Ist das Matrixprodukt kommutativ? Gilt $\det(AB) = \det A \cdot \det B$?

(b) Sei $M = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ eine reelle Matrix mit $\det M = ad - bc \neq 0$. Zeigen Sie, dass $N = \frac{1}{ad-bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$ die Gleichung $MN = I$ erfüllt; I ist die Einheitsmatrix.

(c) Zeigen Sie für jede reelle 2×2-Matrix M, alle $v, w \in \mathbb{R}^2$ und alle $c \in \mathbb{R}$: $M(v+w) = Mv + Mw$ und $M(cv) = c\,(Mv)$. Die Abbildung $v \mapsto Mv$ ist also linear.

(d) R dreht die Ebene um $90°$. Berechnen Sie R^2 und R^4 und deuten Sie die Ergebnisse.

Hinweis: In (b) erst $M \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$ ausrechnen. In (c) genügt eine Komponente: Die i-te Komponente von Mv ist $M_{i1}v_1 + M_{i2}v_2$.

Aufgabe A3 (Lineare Abbildungen und ihr Kern)

Sei $f : \mathbb{R}^n \to \mathbb{R}^m$ linear, also $f(v+w) = f(v) + f(w)$ und $f(cv) = c\,f(v)$ für alle $v, w \in \mathbb{R}^n$ und alle $c \in \mathbb{R}$.

(a) Zeigen Sie $f(0) = 0$ und $f(-v) = -f(v)$ für alle $v \in \mathbb{R}^n$.

(b) Zeigen Sie, dass der Kern $\ker f = \{v \in \mathbb{R}^n \mid f(v) = 0\}$ ein Unterraum von $\mathbb{R}^n$ ist: Er enthält 0 und ist abgeschlossen unter Summen und Vielfachen.

(c) Bestimmen Sie Kern und Bild der Projektion $p : \mathbb{R}^2 \to \mathbb{R}^2$, $p(x, y) = (x, 0)$, und prüfen Sie die Dimensionsformel $\dim \ker p + \dim \mathrm{Bild}\, p = 2$.

(d) Die Verschiebung $g : \mathbb{R} \to \mathbb{R}$, $g(x) = x + 1$, ist nicht linear. Welche Aussage aus (a) verletzt sie? Ist $\{x \in \mathbb{R} \mid g(x) = 0\}$ ein Unterraum von $\mathbb{R}$?

Hinweis: Für $f(0) = 0$: Es ist $f(0) = f(0+0) = f(0) + f(0)$; ziehen Sie auf beiden Seiten $f(0)$ ab. Für $f(-v)$: $-v = (-1)\,v$.

Aufgabe A4 (Induktion)

Zeigen Sie durch vollständige Induktion, dass für alle $n \in \mathbb{N}$ gilt:

(a) $4 \sum_{i=0}^{n} i^3 = n^2(n+1)^2$. Machen Sie vorher die Probe für $n = 3$.

(b) $n + 1 \leq 2^n$.

Aufgabe A5 (Was der Typcheck sieht)

(a) In Lean ist `Set` α definitorisch $\alpha \to \mathrm{Prop}$, und $x \in s$ ist die Anwendung $s\,x$ (Abschnitt 3.6, „Mengen sind Prädikate“). Sei α ein Typ, der als Variable im Kontext steht. Zeigen Sie mit der Regel (App) aus Abschnitt 3.5, dass der Ausdruck $x \in x$ – und damit auch $x \notin x$ – weder im Kontext $x : \alpha$ noch im Kontext $x : \mathrm{Set}\,\alpha$ einen Typ hat. Welcher Term des λ-Kalküls aus Vorlesung 1 scheitert an derselben Stelle?

(b) Einsetzen: Im Kontext $x : \mathbb{N}$ seien $f = x + 2$ und $g = 3 \cdot x$. Berechnen Sie $g[f/x]$ und $f[g/x]$; dabei heißt $t[a/x]$: in t für x den Term a einsetzen. Sind die beiden Ergebnisse definitorisch gleich?

(c) Ist im Kontext $x : \mathbb{N}$ der Term $x + 0$ definitorisch gleich x? Und $0 + x$? Zur Erinnerung: Die Addition auf $\mathbb{N}$ rekursiert über das zweite Argument, $n + 0 \equiv n$ und $n + (m+1) \equiv (n+m) + 1$.

Hinweis: Eine Variable hat den Typ, unter dem sie im Kontext steht, und über (Conv) jeden dazu definitorisch gleichen. Definitorisch gleiche Terme haben dieselbe Normalform (Abschnitt 3.5).

Teil B: In Lean

Die Aufgaben stehen in `lean/Blatt03.lean`. Öffnen Sie die Datei wie auf Blatt 2 in VS Code und ersetzen Sie jedes `sorry` durch einen Beweis; das Infofenster zeigt nach jedem Schritt Ziel und Kontext. Anders als auf Blatt 2 arbeiten Sie jetzt im Taktikmodus: `intro`, `exact`, `apply`, `rw`, `simp`, `ext`, `induction`, `ring`, `decide` und die anderen aus den Vorlesungen 2 und 3. Entstehen mehrere Ziele, etwa nach `constructor` oder `rcases`, bearbeitet Lean sie der Reihe nach; ein Punkt · am Zeilenanfang fokussiert den folgenden eingerückten Block auf das nächste Ziel. Die Sätze bauen aufeinander auf: Was Sie in einer Teilaufgabe beweisen, dürfen Sie in den späteren benutzen – auch solange dort noch ein `sorry` steht.

Aufgabe B1 (Formalisierung von A1)

Die Aussagen aus A1 in Lean. Teil (b) soll ein Beweis im Termmodus werden, ohne `by`; Teil (c) ist in die beiden Inklusionen zerlegt.

Lean `lean/Blatt03.lean`

```
/-- Aufgabe B1 (a): Distributivitaet. -/
theorem distributiv {α : Type*} (s t u : Set α) :
    s ∩ (t ∪ u) = (s ∩ t) ∪ (s ∩ u) := by
  -- ‘ext x‘, ‘constructor‘, dann zerlegen, etwa mit ‘rintro ⟨hs, ht | hu⟩‘
  sorry

/-- Aufgabe B1 (b), im Termmodus: Komplementbildung kehrt Inklusionen um. -/
theorem kompl_antiton {α : Type*} (s t : Set α) (h : s ⊆ t) : tᶜ ⊆ sᶜ :=
  sorry

/-- Aufgabe B1 (c): De Morgan fuer den Durchschnitt, eine Inklusion nach der
anderen. Fuer eine der beiden brauchen Sie ‘Classical.em‘. -/
theorem de_morgan_1 {α : Type*} (s t : Set α) : sᶜ ∪ tᶜ ⊆ (s ∩ t)ᶜ := by
  sorry

theorem de_morgan_2 {α : Type*} (s t : Set α) : (s ∩ t)ᶜ ⊆ sᶜ ∪ tᶜ := by
  sorry

theorem de_morgan {α : Type*} (s t : Set α) : (s ∩ t)ᶜ = sᶜ ∪ tᶜ :=
  -- aus den beiden Inklusionen, mit ‘Set.Subset.antisymm‘
  sorry

/- Worauf beruhen Ihre beiden Beweise? Achten Sie auf ‘Classical.choice‘. -/
#print axioms de_morgan_1
#print axioms de_morgan_2
```

Hinweis: (a) Nach `ext x` ist eine Äquivalenz zu zeigen, `constructor` macht daraus zwei Implikationen. `rintro` zerlegt eine Annahme schon beim Einführen: ein $\wedge$ in seine beiden Teile, ein $\vee$ in zwei Fälle. Die Ziele setzen `Or.inl`, `Or.inr` und der anonyme Konstruktor $\langle \cdot, \cdot \rangle$ zusammen. (b) $x \in t^c$ ist definitorisch $x \in t \to \text{False}$. Ein Beweis von $t^c \subseteq s^c$ ist also eine Funktion, die ein x, einen Beweis von $x \in t^c$ und einen von $x \in s$ nimmt und False liefert. Ein Argument, das im Rumpf nicht vorkommt, schreibt man als `_`; sonst warnt Lean hier vor einer unbenutzten Variablen. (c) `Classical.em` liefert für jede Aussage P einen Beweis von $P \vee \neg P$, `rcases` zerlegt ihn in zwei Fälle. Die Ausgabe von `#print axioms` erscheint im Infofenster. Achten Sie dort auf `Classical.choice`: `Classical.em` ist in Lean ein Satz, und `#print axioms` nennt die Axiome, aus denen er bewiesen ist.

Aufgabe B2 (Formalisierung von A2)

A2 in Lean. In $\mathbb{Z}$ rechnet der Kern Produkte, Determinanten und Potenzen selbst aus. Die Teile (b) und (c) gelten für alle reellen Matrizen; dort vergleicht man Einträge.

Lean lean/Blatt03.lean

```
/- ‘!![a, b; c, d]‘ ist eine 2×2-Matrix, ‘![x, y]‘ ein Vektor in ℝ²; nach
   ‘open Matrix‘ heisst Matrix mal Vektor ‘M *ᵥ v‘. In ℤ rechnet der Kern
   Produkte, Determinanten und Potenzen mit ‘decide‘ selbst aus. In ℝ geht
   das nicht; dort vergleicht man Eintraege. -/
open Matrix

/-- Aufgabe B2 (a): die Rechnungen aus A2 (a). -/
example : !![1, 2; 3, 4] * !![0, 1; 1, 0] = (!![2, 1; 4, 3] : Matrix (Fin 2) (Fin
    2) ℤ) := by
  sorry

example : (!![0, 1; 1, 0] * !![1, 2; 3, 4] : Matrix (Fin 2) (Fin 2) ℤ) ≠
    !![1, 2; 3, 4] * !![0, 1; 1, 0] := by
  sorry

example : (!![1, 2; 3, 4] : Matrix (Fin 2) (Fin 2) ℤ).det = -2 := by
  sorry

/-- Aufgabe B2 (b): die Formel fuer die Inverse aus A2 (b). -/
theorem inverse_2x2 (a b c d : ℝ) (h : a * d - b * c ≠ 0) :
    !![a, b; c, d] * ((a * d - b * c)⁻¹ • !![d, -b; -c, a]) = 1 := by
  -- wie in A2 (b): erst ‘have‘ mit M * !![d, -b; -c, a] = (a * d - b * c) • 1
  sorry

/-- Aufgabe B2 (c): ‘v ↦ M v‘ ist linear (A2 (c)). -/
theorem mulVec_linear (M : Matrix (Fin 2) (Fin 2) ℝ) (v w : Fin 2 → ℝ) (c : ℝ) :
    M *ᵥ (v + w) = M *ᵥ v + M *ᵥ w ∧ M *ᵥ (c • v) = c • (M *ᵥ v) := by
  sorry

/-- Aufgabe B2 (d): die Drehung ‘R‘ aus A2 (d), viermal hintereinander. -/
example : (!![0, -1; 1, 0] : Matrix (Fin 2) (Fin 2) ℤ) ^ 4 = 1 := by
  sorry
```

Hinweis: (a) und (d): `decide`. (b) Halten Sie mit `have` die Gleichung $M \cdot \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} = (ad - bc) \cdot I$ fest: `Matrix.mul_fin_two` rechnet das Produkt aus, `ext i j` vergleicht die Einträge, `fin_cases` geht die Indizes durch, danach `simp` und `ring`. Den Rest erledigen `Matrix.mul_smul`, `smul_smul`, `inv_mul_cancel₀` und `one_smul`. (c) `constructor`, dann für jede Hälfte `ext i`, `simp` mit `Matrix.mulVec`, `dotProduct` und `Fin.sum_univ_two`, und `ring`.

Aufgabe B3 (Formalisierung von A3)

A3 in Lean. Die Abbildung f ist eine gewöhnliche Funktion von $\mathbb{R}^n$ nach $\mathbb{R}^m$, also von `Fin n → ℝ` nach `Fin m → ℝ`; die Linearität steht als Voraussetzungen `hadd` und `hsmul` daneben. Den Kern bauen Sie gebündelt: als Element von `Submodule ℝ (Fin n → ℝ)`, also Trägermenge und drei Beweise. Teil (c) ist die Verschiebung aus A3 (d).

Lean lean/Blatt03.lean

```
/- In Mathlib waere ‘f‘ ein Element von ‘(Fin n → ℝ) →ₗ[ℝ] (Fin m → ℝ)‘. Hier
   ist ‘f‘ eine gewoehnliche Funktion von ℝ^n nach ℝ^m; die Linearitaet steht
   als Voraussetzungen ‘hadd‘ und ‘hsmul‘ daneben. -/

/-- Aufgabe B3 (a): ‘f‘ bildet ‘0‘ auf ‘0‘ ab ... -/
theorem lin_null {n m : ℕ} (f : (Fin n → ℝ) → (Fin m → ℝ))
    (hadd : ∀ v w, f (v + w) = f v + f w) : f 0 = 0 := by
  -- wie in A3 (a): erst ‘have‘ mit f 0 + f 0 = f 0 + 0, dann ‘add_left_cancel‘
  sorry
```

```
/-- ... und ‘-v‘ auf ‘-f v‘. -/
theorem lin_neg {n m : ℕ} (f : (Fin n → ℝ) → (Fin m → ℝ))
    (hsmul : ∀ (c : ℝ) v, f (c • v) = c • f v) (v : Fin n → ℝ) : f (-v) = -f v :=
    by
  sorry

/-- Aufgabe B3 (b): Der Kern als Unterraum, gebuendelt: Traegermenge und
drei Beweise. ‘v ∈ {v | f v = 0}‘ ist definitorisch ‘f v = 0‘;
‘rw [Set.mem_setOf_eq] at hv‘ macht das fuer ‘rw‘ sichtbar. -/
def kern {n m : ℕ} (f : (Fin n → ℝ) → (Fin m → ℝ))
    (hadd : ∀ v w, f (v + w) = f v + f w)
    (hsmul : ∀ (c : ℝ) v, f (c • v) = c • f v) : Submodule ℝ (Fin n → ℝ) where
  carrier := {v | f v = 0}
  zero_mem' := by
    sorry
  add_mem' := by
    sorry
  smul_mem' := by
    sorry

/-- Aufgabe B3 (c): Die Verschiebung aus A3 (d) ist nicht additiv. -/
theorem verschiebung_nicht_linear : ∃ g : ℝ → ℝ, ¬ ∀ x y, g (x + y) = g x + g y :=
  sorry
```

Hinweis: (a) Mit `hadd` rückwärts umgeschrieben, wird aus $f(0)+f(0)$ der Term $f(0+0)$; `add_left_cancel` kürzt. Für das Negative schreibt `neg_one_smul` den Vektor $-v$ als $(-1)\cdot v$. (b) Das Ziel $v \in \{v \mid f(v) = 0\}$ ist definitorisch $f(v) = 0$; `Set.mem_setOf_eq` macht das für `rw` sichtbar. Dazu `add_zero` und `smul_zero`. (c) Zeuge ist $x \mapsto x + 1$ als `fun`-Ausdruck. Setzen Sie $x = y = 0$ ein; `norm_num` erkennt $1 = 2$ als falsch.

Aufgabe B4 (Formalisierung von A4 und A5 (c))

(a) ist A4 (a), (b) die Probe für $n = 10$, (c) ist A4 (b), (d) stellt die Fragen aus A5 (c) als Gleichungen zwischen Funktionen. Das `=` in Lean ist die propositionale Gleichheit. Ob die Terme definitorisch gleich sind, wie A5 (c) fragt, zeigt nur `rfl`; wo es scheitert, hält das `#check_failure` fest.

Lean `lean/Blatt03.lean`

```
/-- Aufgabe B4 (a): die Summe der dritten Potenzen. -/
theorem summe_kuben (n : ℕ) :
    4 * (∑ i ∈ Finset.range (n + 1), i ^ 3) = n ^ 2 * (n + 1) ^ 2 := by
  sorry

/- Aufgabe B4 (b): die Probe fuer ‘n = 10‘. ‘#eval‘ rechnet nur aus; ein
   Beweis, den der Kern durch Rechnen prueft, steht darunter. -/
#eval 4 * (∑ i ∈ Finset.range 11, i ^ 3)

example : 4 * (∑ i ∈ Finset.range 11, i ^ 3) = 10 ^ 2 * 11 ^ 2 := by
  sorry

/-- Aufgabe B4 (c): die Ungleichung aus A4 (b). -/
theorem zweierpotenz (n : ℕ) : n + 1 ≤ 2 ^ n := by
  sorry

/- Aufgabe B4 (d), zu A5 (c). ‘Nat.add‘ rekursiert ueber das zweite Argument.
   Versuchen Sie in beiden Faellen zuerst ‘rfl‘. -/
example : (fun x : ℕ => x + 0) = (fun x => x) :=
  sorry
```

```
example : (fun x : ℕ => 0 + x) = (fun x => x) :=
  sorry

/- Nicht definitorisch gleich: ‘#check_failure‘ erwartet, dass das scheitert. -/
#check_failure (rfl : (fun x : ℕ => 0 + x) = (fun x => x))
```

Hinweis: (a) geht wie die Summe der ungeraden Zahlen in der Vorlesung: `induction n with`, im Schritt `Finset.sum_range_succ` (letzten Summanden abspalten), wegen des Faktors 4 dazu `Nat.mul_add` (ausmultiplizieren), die Induktionsvoraussetzung `ih` und `ring`. (b) Lassen Sie den Kern rechnen: `decide`. (c) Wieder `induction n with`; im Schritt macht `pow_succ` aus 2^{k+1} das Produkt $2^k \cdot 2$, und danach genügt `omega` mit `ih`. (d) Wo `rfl` scheitert, hilft `funext` zusammen mit `Nat.zero_add`.